

MODULE 1

BASICS OF COMPUTER HARDWARE & SOFTWARE

CO - Students will be able to identify hardware & software parts of a computer system.



Prepared By Mr. EBIN PM, AP, IESCE

1

BASICS OF COMPUTER ARCHITECTURE

- Computer is an **electronic machine** that makes performing any task very easy.
- In computer, the CPU executes each instruction provided to it, in a series of steps, this series of steps is called **Machine Cycle**, and is repeated for each instruction.
- **One machine cycle involves**
 - *Fetching of instruction*
 - *Decoding the instruction*
 - *Operand fetching*
 - *Executing the instruction*

Prepared By Mr. EBIN PM, AP, IESCE

EDULINE

2

➤ Computer system has **five basic units** that help the computer to perform operations, which are given below:

1. Input Unit 2. Output Unit 3. Storage Unit
4. Arithmetic Logic Unit 5. Control Unit

❖ **Input Unit**

- Input unit connects the external environment with internal computer system.
- It provides data and instructions to the computer system.
- Commonly used input devices are *keyboard, mouse, magnetic tape* etc.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

3

➤ Input unit performs following tasks:

- Accept the data and instructions from the outside environment.
- Convert it into machine language.
- Supply the converted data to computer system.

❖ **Output Unit**

- It connects the internal system of a computer to the external environment.
- It provides the *results of any computation*, or instructions to the outside world.
- Some output devices are *printers, monitor* etc.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

4

❖ Storage Unit

- This unit holds the data and instructions.
- It also stores the intermediate results before these are sent to the output devices.
- It also stores the data for later use. The storage unit of a computer system can be divided into two categories:

a) Primary Storage:

- This memory is used to store the data which is being currently executed.
- It is used for temporary storage of data. The data is lost, when the computer is switched off.
- **RAM** is used as primary storage memory.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

5

b) Secondary Storage:

- In addition to RAM, every computer also has another storage drive that's used for storing information on a long-term basis, and this is known as secondary storage.
- Any file you create or download is saved to the computer's secondary storage.
- There are two types of storage device used as secondary storage in computers: **HDD (Hard Disk Drive)** and **SSD (Solid-State Drive)**.
- While HDDs are the more traditional of the two, SSDs are fast overtaking HDD as the preferred one for secondary storage.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

6

SSD

- An **SSD** is a storage medium that uses **non-volatile memory** to hold and access data.
- Unlike a hard drive, an **SSD has no moving parts.**

- **Advantages**

- faster access time
- noiseless operation
- higher reliability
- lower power consumption.



Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

7

HDD

- A **hard disk drive** (sometimes abbreviated as a **hard drive**, **HD**, or **HDD**) is a **non-volatile** data storage device.
- It is usually installed internally in a computer, attached directly to the disk controller of the computer's motherboard.



Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

8

❖ Arithmetic Logical Unit (ALU)

- All the **calculations** are performed in ALU of the computer system.
- The ALU can perform basic operations such as **addition, subtraction, division, multiplication** etc.
- Whenever calculations are required, the control unit transfers the data from storage unit to ALU.
- When the operations are done, the result is transferred back to the storage unit.

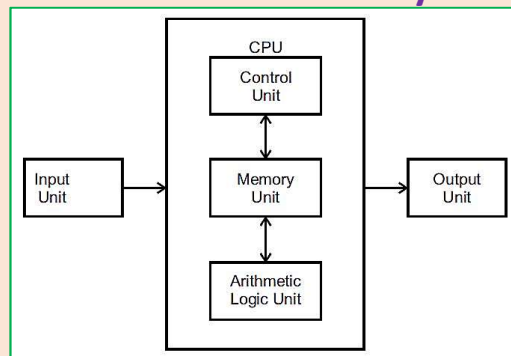
Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

9

❖ Control Unit (CU)

- It controls **all other units** of the computer.
- It **controls the flow of data and instructions** to and from the storage unit to **ALU**.
- Thus it is also known as **central nervous system** of the computer.



Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

10

CPU- CENTRAL PROCESSING UNIT

- A **processor** is an integrated **electronic circuit** that performs the calculations that run a computer.
- A processor performs arithmetical, logical, input/output (I/O) and other basic instructions that are passed from an operating system (OS).
- Most other processes are dependent on the operations of a processor.
- The terms processor, CPU and microprocessor are commonly linked

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

11

- A processor (CPU) is the logic circuitry that responds to and processes the basic instructions that drive a computer.
- The **CPU** is seen as the main and most crucial **integrated circuitry (IC) chip** in a computer, as it is responsible for interpreting most of computers commands.
- CPUs will perform most basic arithmetic, logic and I/O operations, as well as allocate commands for other chips and components running in a computer.



Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

12

- The term *processor* is used interchangeably with the term central processing unit (CPU), although strictly speaking, the **CPU is not the only processor in a computer.**
- The **GPU(graphics processing unit)** is the most notable example, but the hard drive and other devices within a computer also perform some processing independently.
- Nevertheless, the term processor is generally understood to mean the CPU.



(GPU)

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

13

- Processors can be found in PCs, smartphones, tablets and other computers.
- The two main competitors in the processor market are **Intel** and **AMD**.
- CPU is the brain of computer system.
- It performs following tasks:
 - It performs all operations.**
 - It takes all decisions.**
 - It controls all the units of computer.**

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

14

➤ Fetch, Decode, Execute, Store

- Executing a single instruction consists of a particular cycle of events; fetching, decoding, executing and storing.
- For example, to do the add instruction the CPU must
 1. **Fetch** : get the instruction from memory into the processor.
 2. **Decode** : internally decode what it has to do (Eg: add).
 3. **Execute** : take the values from the registers, actually add them together
 4. **Store**: store the result back into another register.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

15

The basic elements of a processor include:

- 1) The arithmetic logic unit (ALU), which carries out arithmetic and logic operations on the operands in instructions.
- 2) The floating point unit (FPU), also known as a math coprocessor or numeric co-processor, a specialized co-processor that manipulates numbers more quickly than the basic microprocessor circuitry can.
- 3) Registers, which hold instructions and other data. Registers supply operands to the ALU and store the results of operations.
- 4) L1 and L2 cache memory. Their inclusion in the CPU saves time compared to having to get data from random access memory (RAM).

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

16

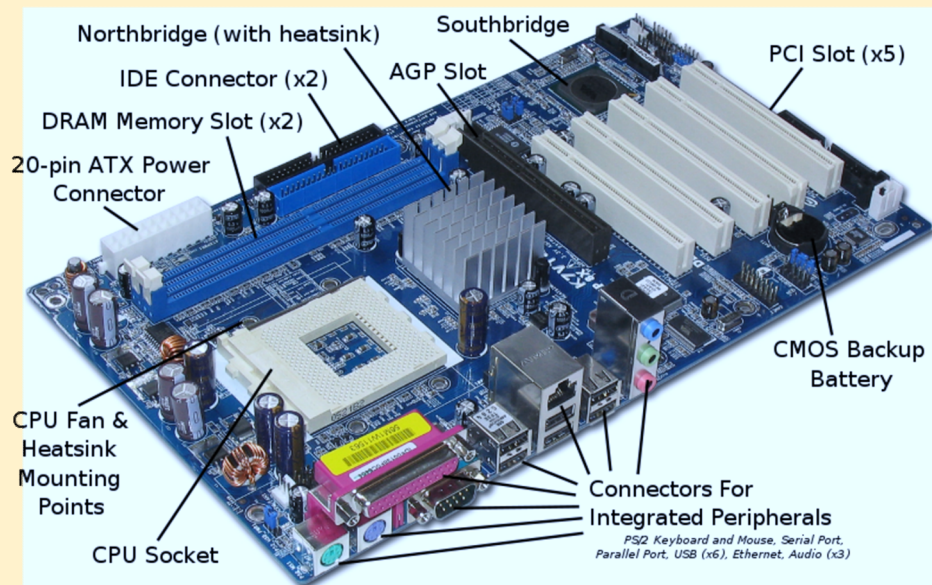
- Most processors today are **multi-core**, which means that the IC contains two or more processors for enhanced performance, reduced power consumption and more efficient simultaneous processing of multiple tasks
- Most computers may have up to **two-four cores**; however, this number can increase up to 12 cores.
- for example, If a CPU can only process a single set of instructions at one time, then it is considered as a single-core processor.
- If a CPU can process two sets of instructions at a time it is called a **dual-core processor**
- four sets would be considered a **quad-core processor**.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

17

❖ MOTHERBOARD



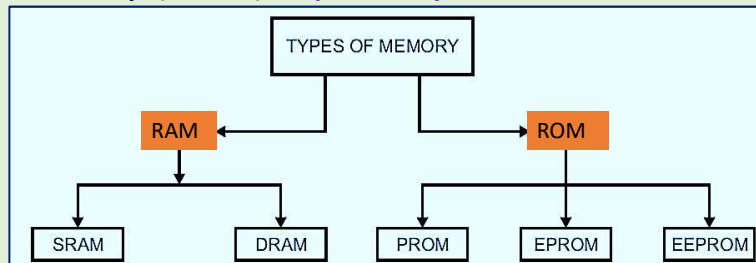
Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

18

TYPES OF MEMORY

- Memory is the most essential element of a computing system because without it computer can't perform simple tasks.
- Computer memory is of **two basic type** - Primary memory(**RAM** and **ROM**) and Secondary memory(hard drive, CD, etc.).
- Random Access Memory (RAM) is **primary-volatile memory** and Read Only Memory (ROM) is **primary-non-volatile memory**.



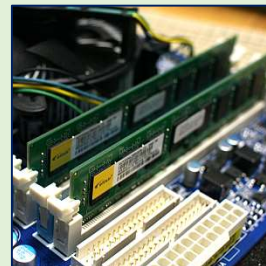
Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

19

❖ RAM - Random Access Memory

- The term RAM is short for Random Access Memory.
- The RAM is where the data the computer is working on is stored while the computer is running.
- Actually, the program is also loaded into RAM before being executed.
- Thus both program and data is stored in RAM while the program is being executed.



Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

20

- The RAM is typically cleared whenever the computer is reset or shutdown.
- Thus, data stored in RAM does not survive computer restarts.

DRAM	SRAM
Constructed of tiny capacitors that leak electricity	Constructed of circuits similar to D Flip-Flop
Requires a recharge every few milliseconds to maintain its data	Holds its contents as long as power is available
Inexpensive	Expensive
Slower than SRAM	Faster than DRAM
Can store many bits per chip	Cannot store many bits per chip
Use less power	Use more power
Generates Less heat	Generates more heat
Used for main memory	Used for cache

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

21

❖ ROM - Read Only Memory

- Stores crucial information essential to operate the system, like the **program essential to boot** the computer.
- It is **not volatile**.
- Always retains its data.
- Used in embedded systems or where the programming needs no change.
- Used in calculators and peripheral devices.
- ROM is further classified into 4 types - **ROM**, **PROM**, **EPROM**, and **EEPROM**.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

22

➤ Types of Read Only Memory

- **PROM (Programmable read-only memory)** - It can be programmed by user. Once programmed, the data and instructions in it cannot be changed.
- **EPROM (Erasable Programmable read only memory)** - It can be reprogrammed. To erase data from it, expose it to **ultra violet light**. To reprogram it, erase all the previous data.
- **EEPROM (Electrically erasable programmable read only memory)** - The data can be erased by applying **electric field**, no need of ultra violet light. We can erase only portions of the chip.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

23

❖ Hard Disk

- The hard disk is a device that can store data and have the data survivor power-off and resets of the computer.
- Data stored on a hard disk can be read at a later point, no matter how longtime the computer has been powered off - provided that the computer and hard disk is still working at the time it is again powered on.



Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

24

- The hard disk is where you normally store your documents and files.
- It is also on the hard disk that the computer's **operating system is stored**, and where the programs you have installed on the computer are stored.
- The hard disk is also where your own programs are stored.
- When the computer is told to execute a program, it loads the program from the hard disk into RAM and executes it from there.
- Programs can also read from and write to the hard disk.
- For instance, the program Microsoft Word can read and write Word documents from and to the hard disk

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

25

HDD

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

26

INPUT OUTPUT DEVICES

- An input device sends information to a computer system for processing, and an output device reproduces or displays the results of that processing.
- Input devices only allow for input of data to the computer.
- Devices are only input devices or output devices, as they can only accept data input from a user or output data generated by a computer.
- However, some devices can accept input and display output, and they are referred to as I/O devices (input/output devices).

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

27

❖ Input devices

- An input device can send data to another device, but it cannot receive data from another device. Examples of input devices include the following.
- **Keyboard and Mouse** - Accepts input from a user and sends that data (input) to the computer. They cannot accept or reproduce information (output) from the computer.
- **Microphone** - Receives sound generated by an input source, and sends that sound to a computer.
- **Webcam** - Receives images generated by whatever it is pointed at (input) and sends those images to a computer.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

28

❖ Output devices

- An output device can receive data from another device and generate output with that data, but it cannot send data to another device. Examples of output devices include the following.
- **Monitor** - Receives data from a computer (output) and displays that information as text and images for users to view. It cannot accept data from a user and send that data to another device.
- **Projector** - Receives data from a computer (output) and displays, or projects, that information as text and images onto a surface, like a wall or a screen. It cannot accept data from a user and send that data to another device.
- **Speakers** - Receives sound data from a computer and plays the sounds for users to hear. It cannot accept sound generated by users and send that sound to another device.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

29

❖ Input/output devices

- An input/output device can receive data from users, or another device (input), and send data to another device (output).
- Examples of input/output devices include the following.
- **CD-RW drive and DVD-RW drive** - Receives data from a computer (input), to copy onto a writable CD or DVD. Also, the drive sends data contained on a CD or DVD (output) to a computer.
- **USB flash drive** - Receives, or saves, data from a computer (input).
- Also, the drive sends data to a computer or another device (output).

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

30

SYSTEM SOFTWARES

❖ Software

- Software is a **collection of instructions** that enable the user to interact with a computer , its hardware or perform tasks
- Without software, most computers would be useless. For example, without your Internet browser software, you could not surf the Internet. Without an operating system, the browser could not run on your computer.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

31

There are two types of software

1. System Software

2. Application Software

➤ **Examples of system software** are Operating System, Compilers, Interpreter, Assemblers, etc.

➤ **Examples of Application software** are Railways Reservation Software, Microsoft Office Suite Software, Microsoft Word, Microsoft PowerPoint , etc.



System Software's



Application software's

Prepared By Mr. EBIN P.M , AP CSE,IESCE

EDULINE

32

❖ SYSTEM SOFTWARES

- Systems software includes the programs that are dedicated to managing the computer itself, such as the operating system and disk operating system (or DOS).
- System software is a software that provides platform to other software's.
- Some examples can be operating systems, antivirus software, disk formatting software, Computer language translators etc.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

33

- These are commonly prepared by the computer manufacturers.
- These software's consists of programs written in low-level languages, used to interact with the hardware at a very basic level.
- System software serves as the interface between the hardware and the end users.
- The most important features of system software include:
 1. Closeness to the system
 2. Fast speed
 3. Difficult to manipulate
 4. Written in low level language
 5. Difficult to design

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

34

➤ Operating System

- An operating system (OS) is a type of system software that **manages computer's hardware and software resources**.
- It provides common services for computer programs.
- An OS acts a link between the software and the hardware.
- It controls and **keeps a record** of the execution of all other programs that are present in the computer, including application programs and other system software.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

35

➤ Compiler:

- A compiler is a software that **translates** the code written in one language to some other language without changing the meaning of the program.
- The compiler is also said to make the target code efficient and optimized in terms of time and space
- A compiler performs almost all of the following operations during compilation: preprocessing, lexical analysis, parsing, semantic analysis (syntax-directed translation), conversion of input programs to an intermediate representation, code optimization and code generation.
- Examples of compiler may include **gcc (C compiler) , g++ (C++ Compiler), javac (Java Compiler)** etc.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

36

➤Interpreter:

- An interpreter is a computer program that **directly executes**, i.e. it performs instructions written in a programming or scripting language.
- Interpreter do not require the program to be previously compiled into a machine language program.
- An interpreter translates high-level instructions into an intermediate form, which is then executes.
- Interpreters are **fast** as it does not need to go through the compilation stage during which machine instructions are generated.
- Interpreter continuously translates the program until the first error is met.
- If an error comes it stops executing. Hence **debugging is easy**. Examples may include Ruby, Python, PHP etc.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

37

➤Assembler:

- An assembler is a program that **converts assembly language into machine code**.
- It takes the basic commands and operations and converts them into binary code specific to a type of processor.
- Assemblers **produce executable code** that similar to compilers. However, assemblers are more simplistic since they only convert low-level code (assembly language) to machine code.
- Since each assembly language is designed for a specific processor, assembling a program is performed using a simple one-to-one mapping from assembly code to machine code.
- On the other hand, compilers must convert generic high-level source code into machine code for a specific processor.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

38

- A **device driver** controls a particular type of device that is attached to your computer, such as a keyboard or a mouse.
- The driver program converts the more general input/output instructions of the operating system to messages that the device type can understand.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

39

HIGH LEVEL & LOW LEVEL LANGUAGES

- Both High level language and low level language are the programming language's types.
- The main difference between high level language and low level language is that, Programmers can easily understand or interpret or compile the high level language in comparison of machine.
- On the other hand, Machine can easily understand the low level language in comparison of human beings.
- Examples of high level languages are C, C++, Java, Python, etc.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

40

❖ Advantages of high-level languages

- High-level language programs are **easy to get developed**.
- It is **easy to visualize** the function of the program.
- The programmer may not remain aware about the architecture of the hardware. So people with out hardware knowledge can also do high level language programming.
- The same high level language program works on any other computer, provided the respective compiler is available for the target new architecture. So **high-level languages are portable**.
- Productivity against high level language programming is enormously increased.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

41

❖ Disadvantages of high-level languages

1. A high level language program **can't get executed directly**. It **requires some translator** to get it translated to machine language. There are two types of translators for high level language programs. They are interpreter and compiler.
- **In case of interpreter**, prior execution, each and every line will get translated and then executed.
 - **In case of compiler**, the whole program will get translated as a whole and will create an executable file. And after that, as when required, the executable code will get executed.
 - These translator programs, especially compilers, are huge one and so are quite expensive.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

42

❖ **Difference Between High Level and Low Level Languages**

	High Level Language	Low Level Language
1.	It is programmer friendly language.	It is a machine friendly language.
2.	High level language is less memory efficient.	Low level language is high memory efficient.
3.	It is easy to understand.	It is tough to understand.
4.	It is simple to debug.	It is complex to debug comparatively.
5.	It is simple to maintain.	It is complex to maintain comparatively.
6.	It is portable.	It is non-portable.
7.	It can run on any platform.	It is machine-dependent.
8.	It needs compiler or interpreter for translation.	It needs assembler for translation.
9.	It is used widely for programming.	It is not commonly used now-a-days in programming.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

43

STRUCTURED PROGRAMMING

- A programming approach in which the program is made as a single structure.
- It means that the **code will execute the instruction by instruction** one after the other.
- It **doesn't support** the possibility of jumping from one instruction to some other with the help of any statement like **GOTO**, etc. Therefore, the instructions in this approach will be executed in a serial and structured manner.
- The languages that **support Structured programming** approach are:
C, C++, JAVA, C# etc

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

44

- The structured program mainly consists of three types of elements:
 - Selection Statements
 - Sequence Statements
 - Iteration Statements
- The structured program consists of well-structured and separated modules. But the entry and exit in a structured program is a single-time event.
- It means that the program uses single-entry and single-exit elements. Therefore a structured program is well maintained, neat and clean program.
- This is the reason why the Structured Programming Approach is well accepted in the programming world

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

45

❖ Advantages of Structured Programming Approach:

- Easier to read and understand
- User Friendly
- Easier to Maintain
- Mainly problem based instead of being machine based
- Development is easier as it requires less effort and time
- Easier to Debug
- Machine-Independent, mostly.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

46

❖ Disadvantages of Structured Programming Approach:

- Since it is Machine-Independent, So it takes time to convert into machine code.
- The converted machine code is not the same as for assembly language.
- The program depends upon changeable factors like data-types. Therefore it needs to be updated with the need on the go.
- Usually the **development** in this approach **takes longer time** as it is language-dependent. Whereas in the case of assembly language, the development takes lesser time as it is fixed for the machine.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

47

- One of the most important concepts of programming is the ability to control a program so that different lines of code are executed or that some lines of code are executed many times.
- The mechanisms that allow us to control the flow of execution are called control structures.
- There are three main categories of control structures
 - **Sequence** - Simply do one instruction then the **next and the next**.
 - **Selection** - This is where you select or choose between two or more flows. The **choice** is decided by asking some sort of question. The answer determines the path (or which lines of code) will be executed.
 - **Iteration** - Also known as **repetition**, it allows some code (one to many lines) to be executed (or repeated) several times.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

48

ALGORITHM & FLOWCHART

- **Algorithm** - An algorithm is the **list of instructions and rules** that a computer needs to do to complete a task.
 - Therefore algorithms are simply a series of instructions that are followed, step by step, to do something useful or [solve a problem](#).
- **Flow chart** is a **graphical representation** of an algorithm.
 - It is a **program-planning tool** to solve a problem. It makes use of symbols which are connected among them to indicate the flow of information and processing.
 - The process of drawing a flowchart for an algorithm is known as **flowcharting**.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

49

No.	Algorithm	Flowchart
1.	Algorithm is step by step procedure to solve the problem.	Flowchart is a diagram created by different shapes to show the flow of data.
2.	Algorithm is complex to understand.	Flowchart is easy to understand.
3.	In algorithm plain text are used.	In flowchart, symbols/shapes are used.
4.	Algorithm is easy to debug.	Flowchart it is hard to debug.
5.	Algorithm is difficult to construct.	Flowchart is simple to construct.
6.	Algorithm does not follow any rules.	Flowchart follows rules to be constructed.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

50

❖ Algorithm has the following characteristics

- **Input:** An algorithm may or may not require input
- **Output:** Each algorithm is expected to produce at least one result
- **Definiteness:** Each instruction must be clear and unambiguous.
- **Finiteness:** If the instructions of an algorithm are executed, the algorithm should terminate after finite number of steps

Flowchart is diagrammatic/Graphical representation of sequence of steps to solve a problem. To draw a flowchart following standard symbols are use

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

51

Symbol Name	Symbol	function
Oval		Used to represent start and end of flowchart
Parallelogram		Used for input and output operation
Rectangle		Processing: Used for arithmetic operations and data-manipulations
Diamond		Decision making. Used to represent the operation in which there are two/three alternatives, true and false etc
Arrows		Flow line Used to indicate the flow of logic by connecting symbols
Circle		Page Connector
		Off Page Connector
		Predefined Process / Function Used to represent a group of statements performing one processing task.
		Preprocessor
		Comments

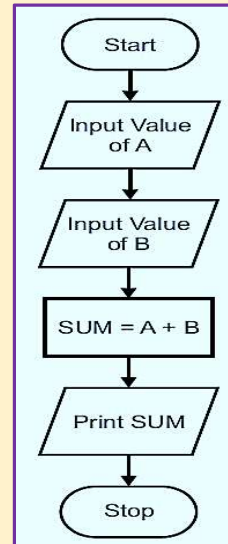
Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

52

Example-1: Algorithm & Flowchart to find the sum of two numbers**Algorithm**

- Step-1 Start
 Step-2 Input first number say A
 Step-3 Input second number say B
 Step-4 $SUM = A + B$
 Step-5 Display SUM
 Step-6 Stop

Flowchart

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

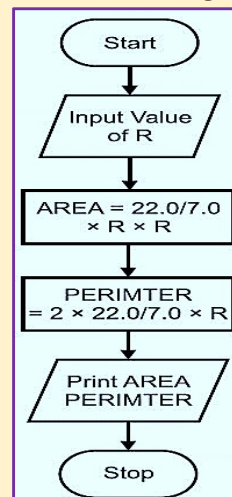
53

Example -2: Algorithm & Flowchart to find Area and Perimeter of Circle

R : Radius of Circle, **AREA** : Area of Circle, **PERIMETER** : Perimeter of Circle

Algorithm

- Step-1 Start
 Step-2 Input Radius of Circle say R
 Step-3 $Area = 22.0/7.0 * R * R$
 Step-4 $PERIMETER = 2 * 22.0/7.0 * R$
 Step-5 Display AREA, PERIMETER
 Step-6 Stop



Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

54

ALGORITHM – BUBBLE SORT

- Sorting refers to **ordering data** in an increasing or decreasing fashion according to some linear relationship among the data items.
- Bubble sort is a simple sorting algorithm.
- This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order.
- This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$ where n is the number of items.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

55

ALGORITHM

```
begin Bubble_Sort(list)
  for all elements of list
    if list[i] > list[i+1]
      swap(list[i], list[i+1])
    end if
  end for
  return list
end Bubble_Sort
```

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

56

Working

The following image shows the unsorted list

21

6

9

33

3

First Iteration

Step 1)

21

6

9

33

3

The values 21 and 6 are compared to check which one is greater than the other.

21

6

9

33

3

21 is greater than 6, so 21 takes the position occupied by 6 while 6 takes the position that was occupied by 21

6

21

9

33

3

Our modified list now looks like the one above.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

57

Step 2)

6

21

9

33

3

The values 21 and 9 are compared.

6

21

9

33

3

21 is greater than 9, so we swap the positions of 21 and 9

6

9

21

33

3

The new list is now as above

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

58

Step 3)

6921333

The values 21 and 33 are compared to find the greater one.

6921333

The value 33 is greater than 21, so no swapping takes place.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

59

Step 4)

6921333

The values 33 and 3 are compared to find the greater one.

6921333

The value 33 is greater than 3, so we swap their positions.

6921333

The sorted list at the end of the first iteration is like the one above

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

60

Prepared By Mr. EBIN PM, AP

30

Second Iteration

The new list after the second iteration is as follows

6

9

3

21

33

Third Iteration

The new list after the third iteration is as follows

6

3

9

21

33

Fourth Iteration

The new list after the fourth iteration is as follows

3

6

9

21

33

Prepared By Mr.EBIN PM, AP, IESCE EDULINE 61

ALGORITHM – LINEAR SEARCH

Linear Search (Array A, Value x)

Step 1: Set i to 1

Step 2: if $i > n$ then go to step 7

Step 3: if $A[i] = x$ then go to step 6

Step 4: Set i to $i+1$

Step 5: Go to Step 2

Step 6: Print Element x Found at index i and go to step 8

Step 7: Print element not found

Step 8: Exit

Linear Search

10

14

19

26

27

31

33

35

42

44

=
33

Prepared By Mr.EBIN PM, AP, IESCE EDULINE 62

PSEUDO CODE

- It's simply an implementation of an algorithm in the form of **annotations** and **informative text** written in **plain English**.
- It has **no syntax** like any of the programming language and thus **can't be compiled or interpreted by the computer**.
- It's the cooked up representation of an algorithm.
- Pseudo code, as the name suggests, is a **false code** or a representation of code which can be understood by even a layman with some school level programming knowledge

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

63

❖ Advantages of Pseudo code

- Improves the **readability** of any approach. It's one of the best approaches to start implementation of an algorithm.
- Acts as a **bridge between the program and the algorithm** or flowchart. Also works as a rough documentation, so the program of one developer can be understood easily when a pseudo code is written out. In industries, the approach of **documentation** is essential. And that's where a pseudo-code proves vital.
- The main goal of a pseudo code is to **explain what exactly each line of a program should do**, hence making the code construction phase easier for the programmer.

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

64

➤ **An example of pseudo code to create a program to add 2 numbers together and then display the result.**

Start Program

Enter two numbers, A, B

Add the numbers together

Print Sum

End Program

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

65

➤ **Pseudo code for Linear Search**

procedure linear search (list, value)

 for each item in the list

 if match item == value

 return the item's location

 end if

 end for

end procedure

Prepared By Mr.EBIN PM, AP, IESCE

EDULINE

66

etc.

1.3.3 Registers

Registers are a type of computer memory used to quickly accept, store, and transfer data and instructions that are being used immediately by the CPU. The registers used by the CPU are often termed as Processor registers. A processor register may hold an instruction, a storage address, or any data. The computer needs processor registers for manipulating data and a register for holding a memory address. The register holding the memory location is used to calculate the address of the next instruction after the execution of the current instruction is completed.

The most commonly used CPU registers are:

- i. **Data register:** The data register holds the operand read from the memory.
- ii. **Address register:** The main function of address register is to hold the memory location of instruction which is being executed.
- iii. **Instruction register:** This register holds the current instruction so that it can be decoded and input to the control and timing unit. Specifically, the instruction register holds the opcode which defines the type of instruction.
- iv. **Accumulator:** Main function of Accumulator (AC) is to store the output which is generated by our system. When CPU (Center Processing Unit) will execute some instruction then it will produce the result, now AC register is needed to store those produced data.
- v. **Program counter:** Program Counter (PC) register's function is to hold all records in sequence of entire execution of programs. PC has the memory address of further instruction that is fetched in next step. PC registers to keep track the address of

Properties		
Data retention	RAM is a volatile memory which could store the data as long as the power is supplied.	ROM is a non-volatile memory which could retain the data even when power is turned off.
Working type	Data stored in RAM can be retrieved and altered.	Data stored in ROM can only be read.
Use	Used to store the data that has to be currently processed by CPU temporarily.	It stores the instructions required during bootstrap of the computer.
Speed	It is a high-speed memory.	It is much slower than the RAM.
CPU Interaction	The CPU can access the data stored on it.	The CPU cannot access the data stored on it unless the data is stored in RAM.
Size and Capacity	Large size with higher capacity.	Small size with less capacity.
Used as/in	CPU Cache, Primary memory.	Firmware, Micro-controllers
Accessibility	The data stored is easily accessible	The data stored is not as easily accessible as in RAM
Cost	Costlier	Cheaper than RAM.

Table 1.1 RAM vs ROM

1.4.3 Cache Memory

The data and instructions that are required during the processing of data are brought from the secondary storage devices and stored in the RAM. For processing, it is required that the data and instructions are accessed from the RAM and stored in the registers. The time taken to move the data between RAM and CPU registers is large. This affects the speed of processing of computer, and results in decreasing the performance of CPU.

Cache memory is a very high-speed memory placed in between RAM and CPU. Cache memory increases the speed of processing. It is a storage buffer that stores the data that is used more often, temporarily, and makes them available to CPU at a fast rate. During processing, CPU first checks cache for the required data. If data is not found in cache, then it looks in the RAM for data. Cache memory is built into the processor, and may also be located next to it on a separate chip between the CPU and RAM. Cache memory is very expensive, and hence, will be smaller in size.

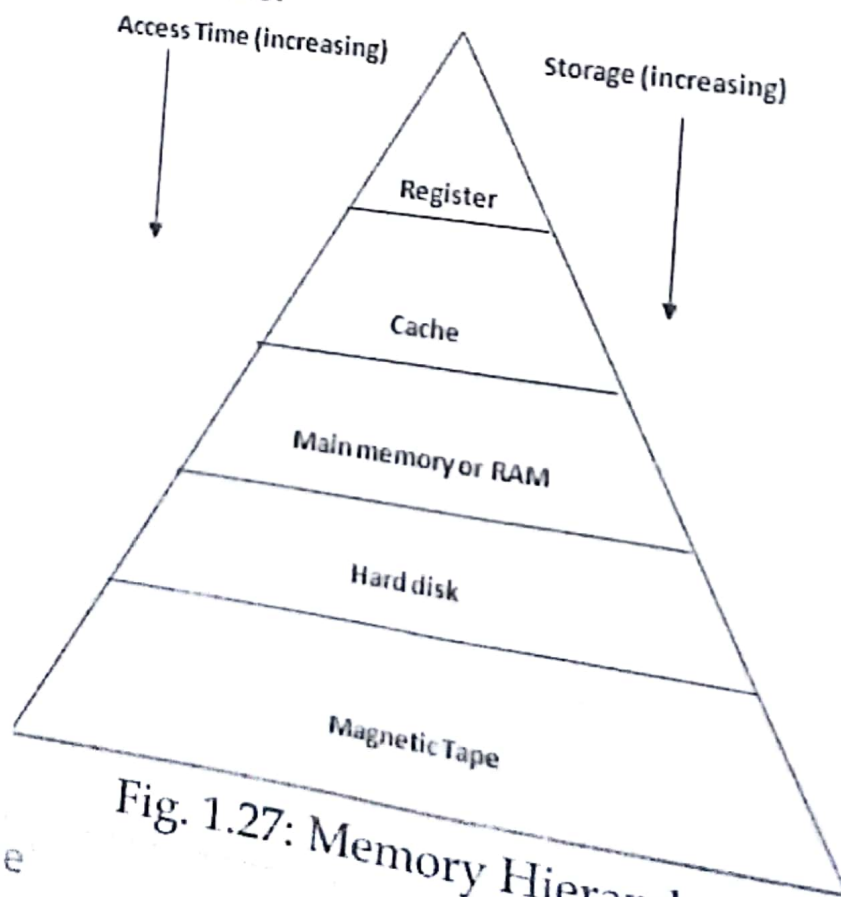


Fig. 1.27: Memory Hierarchy

It relates to computer like processor, input, output devices etc.

System Software	Application Software
System Software maintain the system resources and give the path for application software to run.	Application software is built for specific tasks.
Low level languages are used to write the system software.	While high level languages are used to write the application software.
It's a general-purpose software.	While it's a specific purpose software.
Without system software, system can't run.	While without application software system always runs.
System software runs when system is turned on and stops when system is turned off.	While application software runs as per the user's request.
Example of system software are operating systems	Example of application software are Photoshop, VLC player etc.
System Software programming is complex than application software.	Application software programming is simpler as comparison to system software.

Table 2.1: System Software vs Application software

High Level Language	Low Level Language
It is programmer/user friendly language.	It is a machine friendly language.
High level language is less memory efficient.	Low level language is high memory efficient.
It is easy to understand.	It is tough to understand.
It is simple to debug.	It is complex to debug comparatively.
It is simple to maintain.	It is complex to maintain comparatively.
It is portable.	It is non-portable.
It can run on any platform.	It is machine-dependent.

Table 2.2: HLL vs LLL

COMPILER	INTERPRETER
Compiler looks at the entire source code.	Interpreter looks at a source code line-by-line
Compiler converts the entire source code into object-code and creates the object code. The object code is then executed by the user.	Interpreter converts a line into machine executable form, executes the line, and proceeds with the next line
For a given source code, once it is compiled, the object code is created. This object code can be executed multiple number of times by the user.	Interpreter executes line-by-line, so executing the program using an interpreter means that during each execution, the source code is first interpreted and then executed.
During execution of an object code, the compiler is not required	Both interpreter and the source code is required during execution
Faster compared to Interpretation	Interpreter interprets line-by-line, the interpreted code runs slower than the compiled code.

Table 2.3: Compiler vs Interpreter

Eg 3: Algorithm to find the average of 3 numbers

Step 1: Start

Step 2: Read three numbers a,b,c,

Step 3: Calculate $\text{Sum} = a + b + c$

Step 4: Calculate the average of three numbers,
 $\text{Avg} = \text{Sum} / 3$

Step 5: Print the result, Avg

Step 6: Stop

Eg. 4: Algorithm to print the largest of three numbers

Step 1: Start

Step 2: Read three numbers a, b, c.

Step 3: If $A > B$ then goto step 4 otherwise goto step 5

Step 4: If $A > C$ print 'A is greatest' otherwise print 'C is greatest'.

Step 5: If $B > C$ print 'A is greatest' otherwise print 'C is greatest'.

Step 6: Stop

Eg. 5: Algorithm to check whether the given number is odd or even

Step 1: Start

Step 2: Read the number, a

Step 3: If $a \% 2 = 0$ print 'Even' otherwise print 'Odd'

Step 4: Stop

Eg. 6: Algorithm to find the area of a triangle.

Step 1: Start

Step 2: Read the breadth and height B,H

Step 3: $\text{AREA} = 1/2 * B * H$

Step 4: Display the result, AREA

Step 5: Stop

Eg. 7: Algorithm to find the factorial of a number

Step 1: Start

Step 2: Read the number, n

Step 3: Initialize $i = 1$ and $\text{fact} = 1$

Step 4: Calculate $\text{fact} = \text{fact} * i$
 Step 5: Set $i = i + 1$
 Step 6: Check whether $i \leq n$, If yes goto step 4 otherwise goto step 7
 Step 7: Print the factorial, fact
 Step 8: Stop

Eg. 8: Algorithm to find the sum of digits of a number

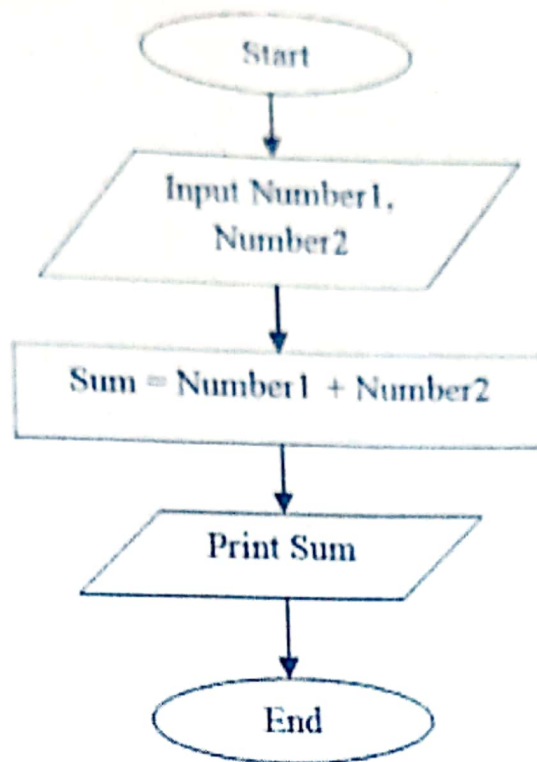
Step 1: Start
 Step 2: Read the number, n
 Step 3: initialize $\text{Sum} = 0$
 Step 4: Repeat the steps 5, 6 and 7 while $n > 0$
 Step 5: Calculate $r = n \% 10$
 Step 6: Calculate $\text{Sum} = \text{Sum} + r$
 Step 7: Set $n = n / 10$ and goto step 4
 Step 8: Print the sum of digits, Sum
 Step 9: Stop

Eg. 9: Algorithm to find the roots of a quadratic equation

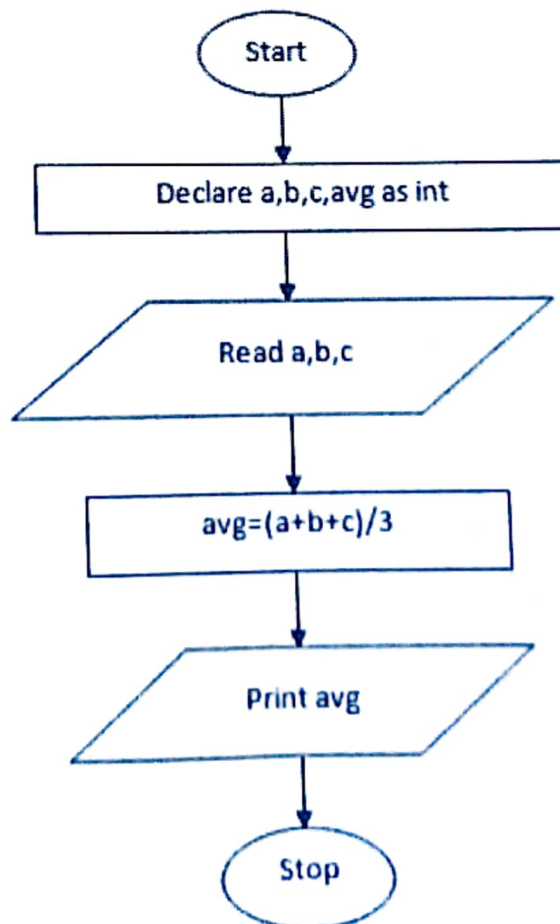
Step 1: Start
 Step 2: Declare variables a, b, c, D, r1, r2, r and i
 Step 3: Read the coefficients a, b & c
 Step 4: Calculate discriminant, $D = b^2 - (4 * a * c)$
 Step 5: Check whether $D > 0$ if true then
 Step 5. 1: Calculate the roots,
 $r1 = (-b + \sqrt{D}) / 2 * a$ and
 $r2 = (-b - \sqrt{D}) / 2 * a$
 Step 5.2: Display the roots r1 and r2.
 Step 5. 3: Go to Step 8
 Step 6: Check whether $D = 0$ if true then go to Step 6.1 otherwise go to Step 7
 Step 6.1: Calculate the root, $r = -b / (2 * a)$
 Step 6.2: Display the root, r and go to Step 8
 Step 7: Calculate real part and imaginary part,
 $rp = -b / 2 * a$ and $ip = \sqrt{-D} / 2a$
 Step 7.1: Display $rp + j(ip)$ and $rp - j(ip)$ as roots
 Step 8: Stop

Flowchart Examples

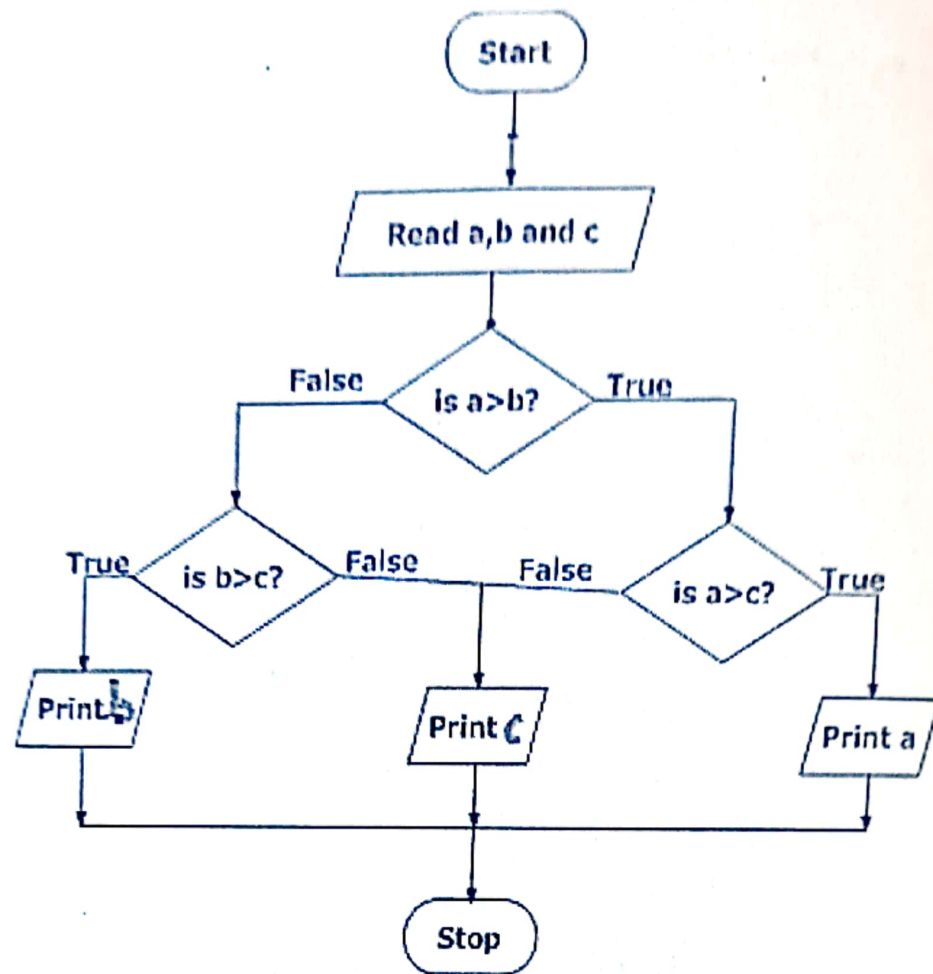
Eg. 1: Flowchart to find the sum of 2 numbers



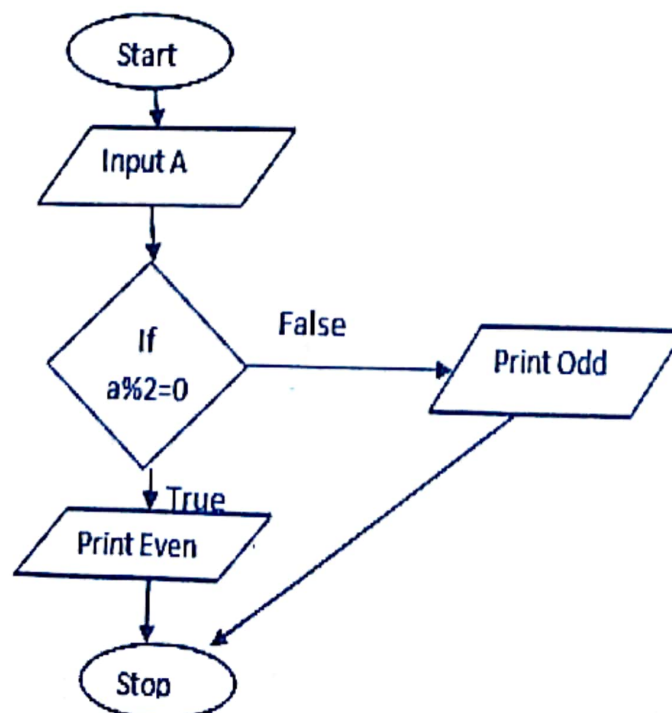
Eg. 2: Flowchart to find the average of 3 numbers.



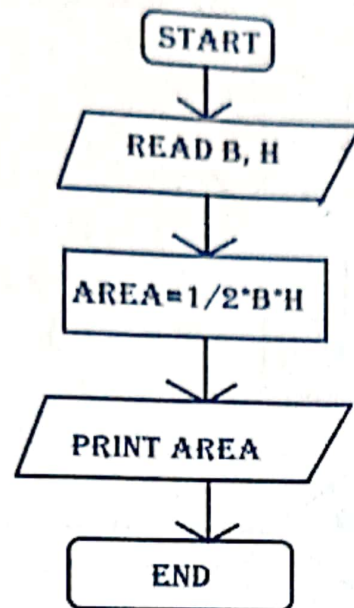
Eg. 3: Flowchart to find the largest among 3 numbers.



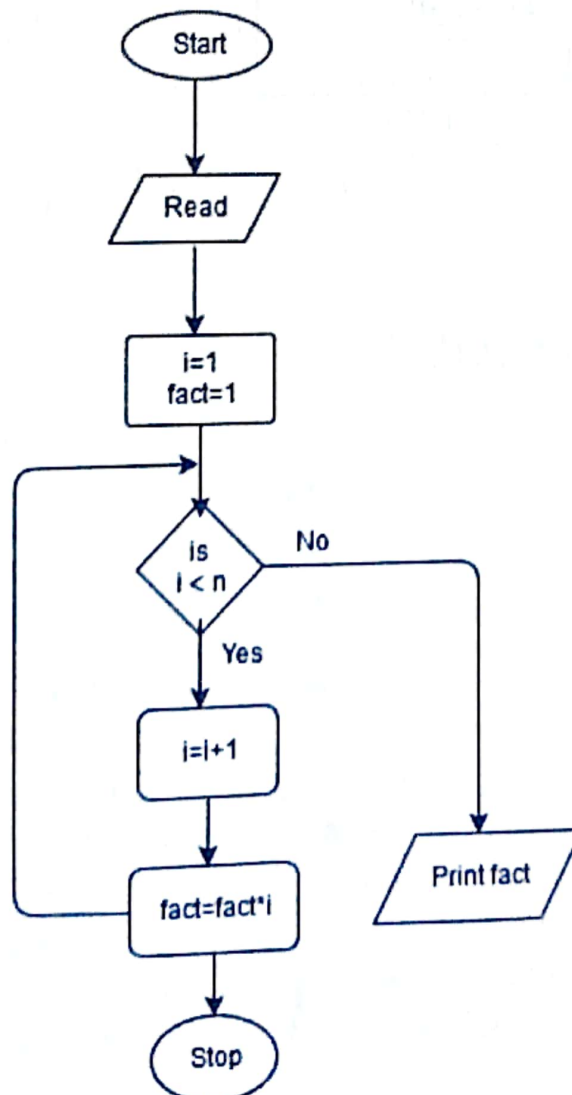
Eg. 4: Flowchart to check whether the given number is odd or even.



Eg. 5: Flowchart to find the area of a triangle.

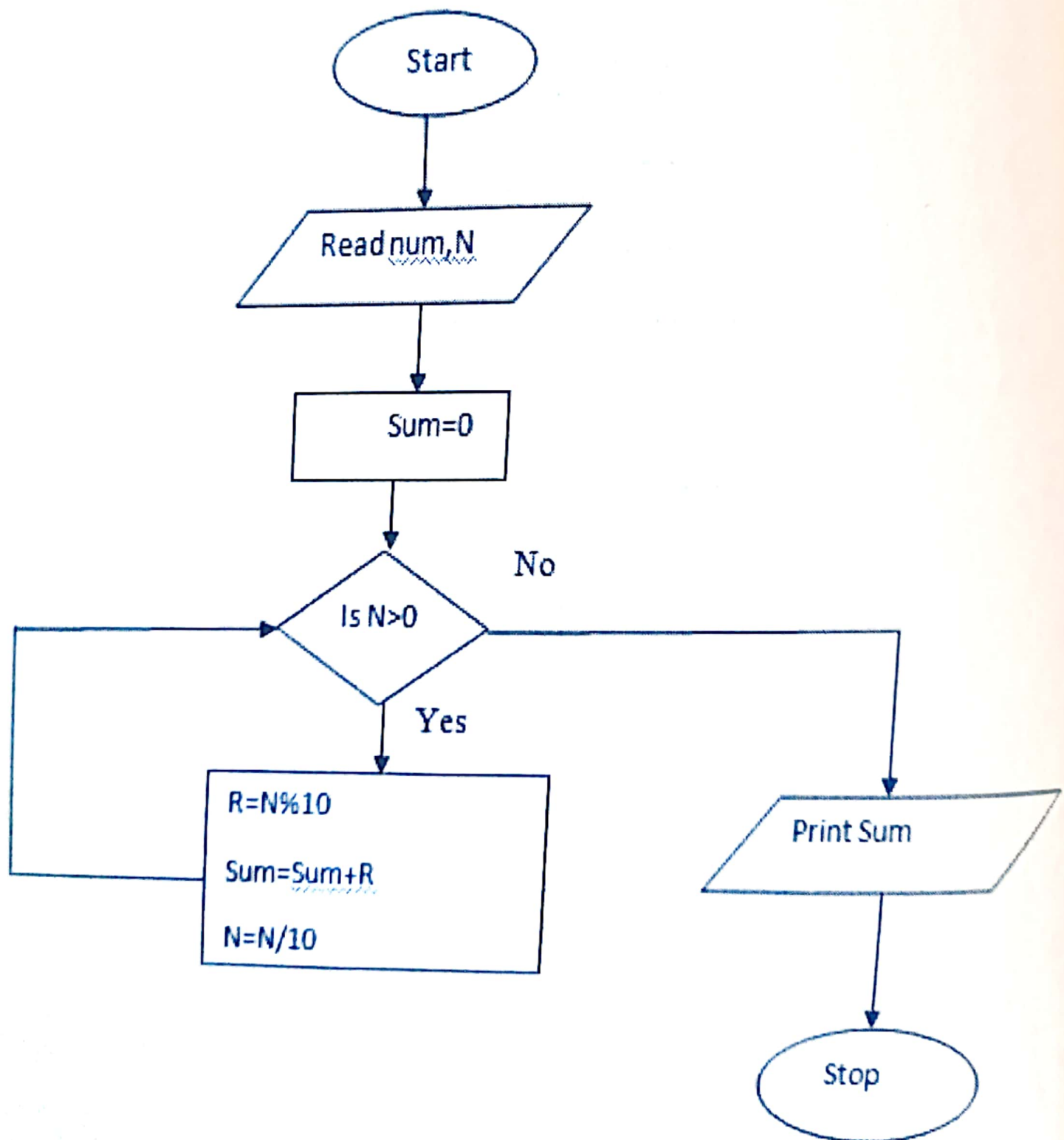


Eg. 6: Flowchart to find the factorial of a number



PROGRAMMING IN C

Eg. 7: Flowchart to find the sum of digits of a number



Pseudo Code Examples

Eg. 1: Pseudocode to find the sum and average of n natural numbers

1. Start
2. Read n
3. $\text{Sum} \leftarrow 0$
4. For $i \leftarrow 1$ to n do
 - Read n_i
 - $\text{Sum} \leftarrow \text{sum} + n_i$
 EndFor
5. $\text{Avg} \leftarrow \text{Sum}/n$
6. Print Sum, Avg
7. Stop

Eg.2: Pseudo code to find the largest of three numbers

1. START
2. READ the values of 3 numbers A, B and C
3. IF the value of $A > \text{value of B}$ THEN
 - GOTO step 4
 - ELSE
 - GOTO step 5
 - ENDIF
4. IF the value of $A > \text{value of C}$ THEN
 - PRINT "The maximum value is that of A"
 - GOTO step 6
 - ELSE
 - GOTO step 5
 - ENDIF
5. IF the value of $B > \text{value of C}$ THEN
 - PRINT "The maximum value is that of B"
 - GOTO step 6
 - ELSE
 - PRINT "The maximum value is that of C"
 - GOTO step 6
 - ENDIF
6. STOP

3.5 BUBBLE SORT

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

We take an unsorted array for our example. Sorted array is the output.



Bubble sort starts with very first two elements, comparing them to check which one is greater.



In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.



We find that 27 is smaller than 33 and these two values must be swapped.



Next we compare 33 and 35. We find that both are in already sorted positions.



Then we move to the next two values, 35 and 10.



We know then that 10 is smaller 35. Hence they are not sorted.

14	27	33	35	10
----	----	----	----	----

We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this

14	27	33	10	35
----	----	----	----	----

After the second iteration, it should look like this -

14	27	10	33	35
----	----	----	----	----

After the third iteration, it should look like this -

14	10	27	33	35
----	----	----	----	----

After last iteration

10	14	27	33	35
----	----	----	----	----

Pseudo Code for Bubble Sort:

Begin

for j=1 to n-1

for i=1 to n-1

if list[i] > list[i+1]

swap (list[i], list[i+1])

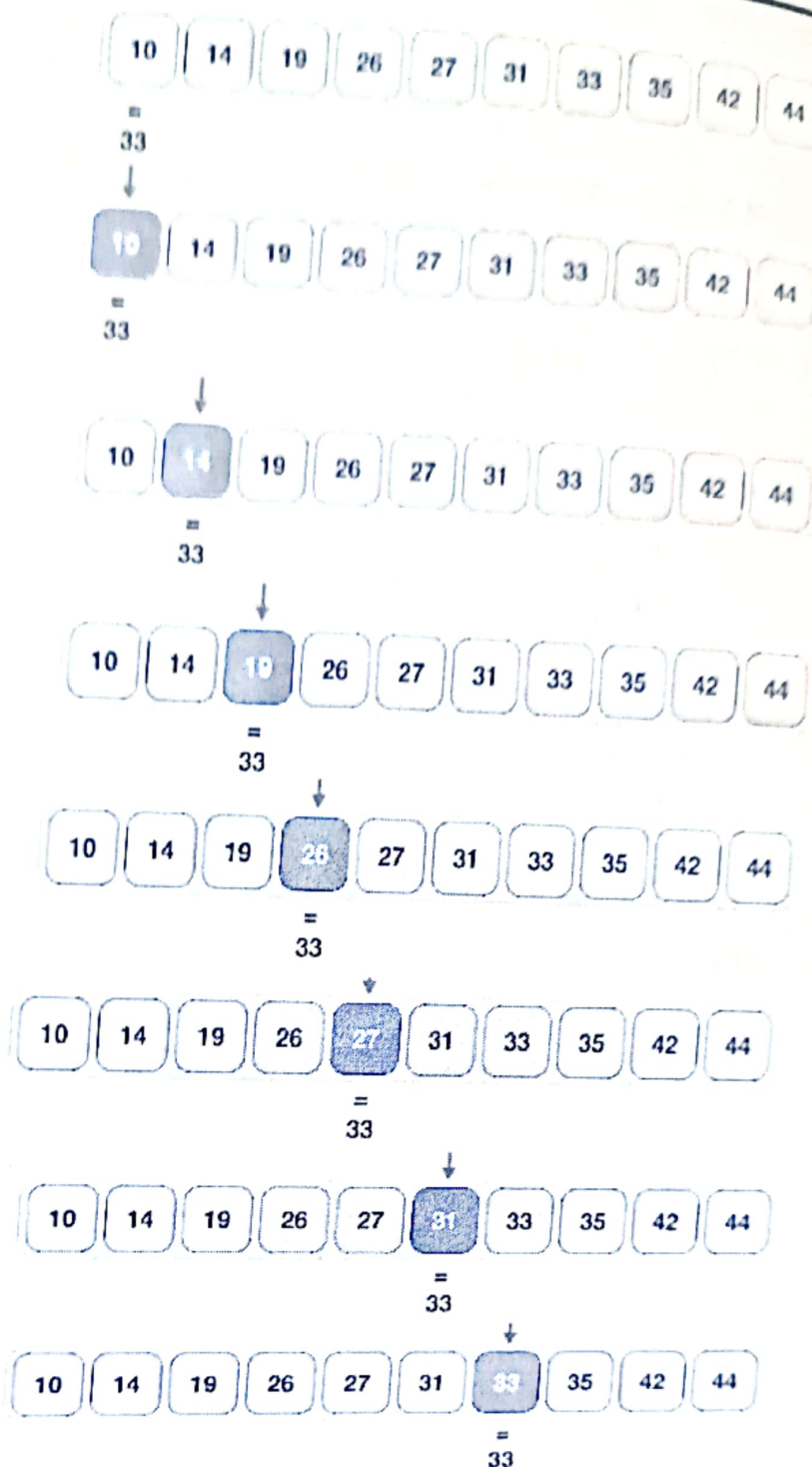
end if

end for

end for

end

Detailed algorithm for Bubble Sort is given in page number a.19 (appendix).



The element is found!

Pseudo code of Linear Search

```

Begin
  Read Search element, e
  Read n
  While (i <= n)
    Read num
    if (num == e)
      Print 'Element Found'
    End if
  End while

```

Detailed algorithm for Linear Search is given in page number a.25 (appendix).

QUESTION BANK

MODULE 1:

1. Differentiate between compiled and interpreted languages with suitable example.
2. Write a short note on the processor of a computer.
3. Describe about any three input devices used in a computer.
4. Write an algorithm and draw a flowchart to calculate the roots of a quadratic equation.
5. Write an algorithm and draw a flowchart to convert the length in feet to centimeter.
6. Write an algorithm and draw a flowchart that will read the two sides of a rectangle and calculate its area.
7. Write a pseudocode for sorting n elements of an array in ascending order.
8. What is a flowchart? What are the features of a flowchart?
9. Write an algorithm to accept the age of a person and check whether he/she is eligible to vote or not.
10. Differentiate between RAM and ROM.
11. Compare and contrast between application software and system software.
12. Differentiate between primary and secondary memory.
13. Write an algorithm and draw a flowchart to find whether a number is prime or not.
14. Write an algorithm and draw a flowchart to display the even numbers upto n .
15. Write an algorithm and draw a flowchart to find the area of a triangle.
16. Write an algorithm and draw a flowchart to print the factorial of a number.
17. Write an algorithm and draw a flowchart to perform linear search on an array of ' n ' elements.

PROGRAMMING IN C

- . Write an algorithm and draw a flowchart to find the square of a number.
- . Write an algorithm and draw a flowchart to find the smallest of three numbers.
- . Write an algorithm and draw a flowchart to calculate HCF.